SISTEMAS EMBEBIDOS

Universidad Politécnica del Bicentenario.

Reporte Practica 4. Servoaccionamiento de Luminosidad.

Ingeniería Mecatronica 701

Edgar Uriel Rodriguez Moreno *24033660*

Jorge Zamorano Chavez *24033247*

Axel Gabriel Domínguez Macias *24032952*

Braulio Vázquez Morado *24032975*

Daisy Jackeline Hernandez Ramos *24033069*

Silao, Guanajuato a 20 de Octubre de 2026.

Introducción

El control eficiente de la energía eléctrica y la regulación de la intensidad luminosa en sistemas de corriente alterna representan pilares fundamentales dentro de la automatización moderna y la mecatrónica aplicada. La capacidad de modular el flujo de energía hacia una carga de potencia mediante el uso de microcontroladores y actuadores mecánicos permite resolver problemas de control de manera precisa y automatizada. El presente reporte documenta de forma exhaustiva el diseño, la integración de hardware, el desarrollo lógico y la validación experimental de un prototipo enfocado en regular la intensidad de un foco mediante un dimmer de 2000W y 127V, el cual es accionado de manera automática mediante el acoplamiento físico de un servomotor SG90 a su perilla de ajuste. El sistema se complementa con una interfaz de usuario compuesta por dos pulsadores para el control direccional del movimiento y una pantalla LCD conectada a un microcontrolador ESP32 DevKit V1 para la visualización en tiempo real de la posición angular, logrando un funcionamiento completamente exitoso donde la intensidad del foco pudo regularse de manera gradual y precisa.

Marco Teórico

El desarrollo de este sistema de control mecatrónico combina principios de electrónica digital, procesamiento de señales y conversión electromecánica. El control de la luz no se realiza mediante circuitos complejos de modulación de fase directos por software en corriente alterna, sino a través de un acoplamiento mecánico indirecto donde un microcontrolador procesa una consigna numérica que se traduce en un movimiento angular exacto capaz de girar la perilla de un dimmer de potencia. Para comprender a profundidad el funcionamiento del prototipo, es necesario analizar cada uno de los componentes de hardware y software involucrados:

Microcontrolador ESP32 DevKit V1: Es una placa de desarrollo compacta y de alto rendimiento basada en el sistema en un chip ESP32 de Espressif Systems,

equipada con conectividad inalámbrica y un microprocesador de 32 bits de doble núcleo. Funciona como la unidad central de procesamiento y el núcleo de toma de decisiones del sistema, encargado de monitorear el estado de los pulsadores, ejecutar las rutinas de control, generar las señales de modulación por ancho de pulso para el servomotor y gestionar la comunicación I2C para actualizar la pantalla LCD. Es compatible con lenguajes como C y C++ mediante el entorno de Arduino o el framework ESP-IDF, así como con MicroPython.

Servomotor SG90: Es un actuador rotativo compacto ampliamente utilizado en robótica y prototipado rápido, compuesto por un motor de corriente continua, un tren de engranajes reductores y un circuito de control interno. Su función en esta práctica consiste en recibir las señales PWM del ESP32 para girar con precisión y mover físicamente la perilla del dimmer dentro de un rango de 0 a 180 grados. Se controla mediante cualquier plataforma que genere señales digitales PWM estándar a una frecuencia de 50 Hz, siendo compatible con lenguajes como C, C++ y Python.

Dimmer; Dispositivo de regulación de potencia diseñado para controlar la cantidad de energía eléctrica suministrada a una carga de corriente alterna mediante el ajuste de su perilla rotativa. En este montaje, el eje del servomotor se acopló directamente a la perilla del dimmer, de modo que al variar el ángulo del servo, el dispositivo modifica la conducción de corriente hacia el foco incandescente, permitiendo transicionar suavemente entre diferentes niveles de iluminación. Al ser un dispositivo accionado mecánicamente en este arreglo, responde directamente al movimiento físico generado por el servomotor.

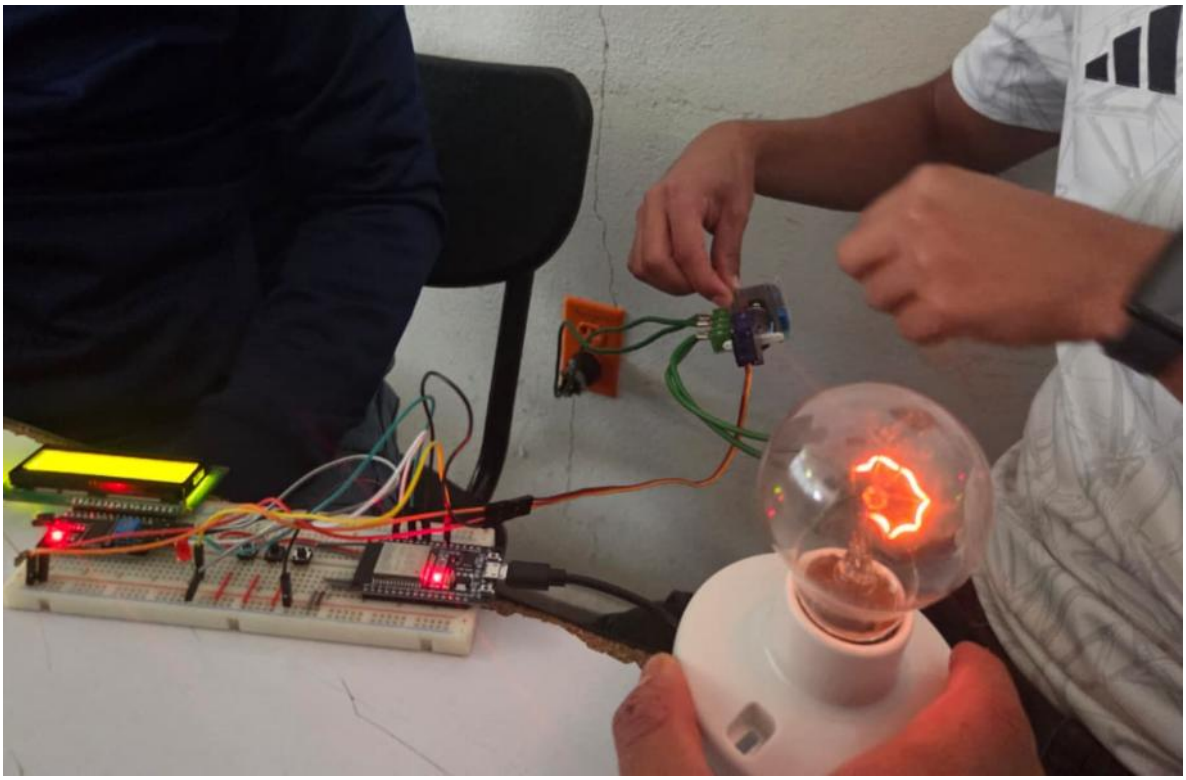
Pantalla LCD con Adaptador I2C: Dispositivo de visualización alfanumérica de cristal líquido capaz de desplegar hasta 16 caracteres por línea en dos filas, equipado con un módulo adaptador basado en el circuito integrado PCF8574 que convierte la interfaz paralela en un bus serial I2C de dos hilos. Funciona como la interfaz visual de salida para el operador, mostrando en tiempo real el valor numérico exacto de la posición angular del servomotor en grados. Es

compatible con C, C++ y Python mediante el uso de librerías especializadas para el controlador HD44780.

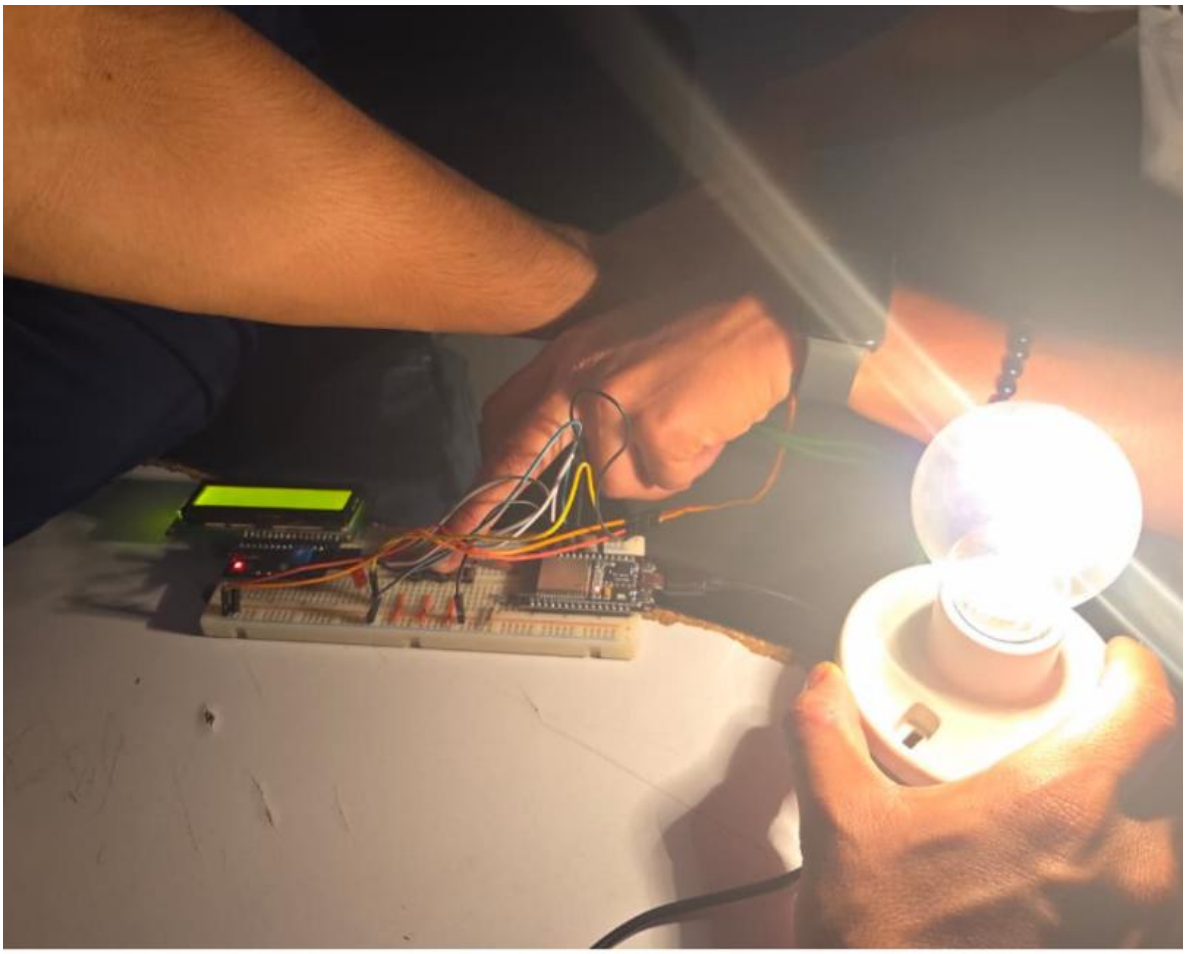
Pulsadores: Interruptores electromecánicos momentáneos de tipo normalmente abierto que establecen un contacto eléctrico temporal al ser presionados. Conforman la interfaz de entrada manual del sistema, utilizándose dos botones independientes: uno dedicado a incrementar el ángulo de rotación hacia adelante y otro enfocado en regresar el servo. Son compatibles con cualquier lenguaje de programación capaz de leer estados lógicos digitales en pines de entrada con resistencias de polarización.

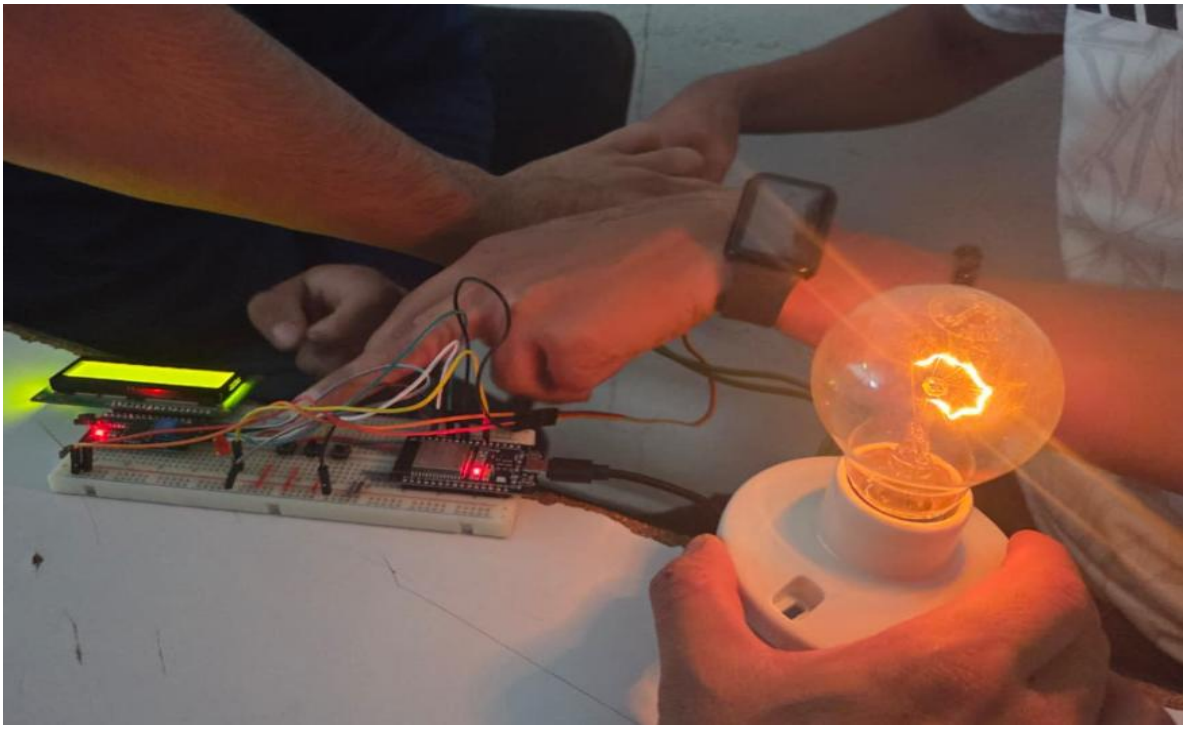
Desarrollo

El desarrollo experimental de la práctica se estructuró mediante la integración física de los elementos y la programación del microcontrolador. En la primera fase se realizó el montaje del circuito sobre una protoboard conectando la pantalla LCD al bus I2C del ESP32 utilizando los pines GPIO 21 para SDA y GPIO 22 para SCL. Los dos pulsadores de control manual se vincularon a los pines digitales configurados internamente con resistencias de pull-up para asegurar lecturas estables. El pin de control del servomotor SG90 se conectó a una salida PWM del microcontrolador para posicionar su eje con exactitud. Mecánicamente, el servomotor se fijó y acopló de manera directa a la perilla del dimmer de 2000W y 127V, logrando que el movimiento de rotación de 0 a 180 grados modificara físicamente la resistencia del dispositivo y regulara la energía entregada al foco incandescente de corriente alterna.



En cuanto a la lógica de software, el firmware implementado en el ESP32 se diseñó para garantizar una respuesta fluida y rápida ante las acciones del usuario. El código evalúa de manera constante el estado de los botones incorporando tiempos de filtrado y rutinas de repetición continua cuando se mantiene presionada una pulsación, lo que permite un incremento o decremento dinámico y veloz del ángulo del servomotor. Cada modificación realizada actualiza de inmediato la posición del actuador, envía los datos correspondientes a la pantalla LCD para reflejar los grados actuales en tiempo real y transmite los valores a través del puerto serial, asegurando una sincronización perfecta entre la interfaz de usuario y la respuesta mecatrónica del sistema.





```
#include <ESP32Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------- Pines ----------
#define BTN_UP    32    // Botón 1: Subir ángulo
#define BTN_DOWN  33    // Botón 2: Bajar ángulo
#define SERVO_PIN 18    // Control del Servo
#define LCD_SDA   21    // LCD I2C SDA
#define LCD_SCL   22    // LCD I2C SCL
#define LCD_ADDR  0x27  // Dirección I2C LCD

// ---------- Objetos ----------
Servo servo;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

// ---------- Constantes y Variables ----------
const int MIN_ANGULO = 0;
const int MAX_ANGULO = 180;

int angulo = 0; // Posición actual del servomotor (0° a 180°)

// ---------- Ajustes para Máxima Velocidad ----------
const unsigned long DEBOUNCE_MS     = 25;
const unsigned long HOLD_INICIO_MS  = 200;  // Inicia repetición
rápida tras 200ms
const unsigned long HOLD_REPETIR_MS = 20;   // Incrementa cada
20ms (Avance ultra rápido)

struct Boton {
  uint8_t pin;
  bool estable;              // HIGH = suelto, LOW = presionado
  bool lecturaAnterior;
  unsigned long ultimoCambio;
  unsigned long inicioPulsacion;
  unsigned long ultimaRepeticion;
};

Boton bUp   = {BTN_UP,   HIGH, HIGH, 0, 0, 0};
Boton bDown = {BTN_DOWN, HIGH, HIGH, 0, 0, 0};

// Función de lectura con aceleración continua
bool leerBoton(Boton &b) {
  bool lectura = digitalRead(b.pin);
  unsigned long ahora = millis();
  bool accion = false;

  if (lectura != b.lecturaAnterior) {
    b.ultimoCambio = ahora;
    b.lecturaAnterior = lectura;
  }

  if ((ahora - b.ultimoCambio) > DEBOUNCE_MS && lectura !=
b.estable) {
    b.estable = lectura;
    if (b.estable == LOW) {          // Presión inicial
      accion = true;
      b.inicioPulsacion = ahora;
      b.ultimaRepeticion = ahora;
```

```
    }
  }

  // Repetición veloz mientras permanezca presionado
  if (b.estable == LOW &&
      (ahora - b.inicioPulsacion) > HOLD_INICIO_MS &&
      (ahora - b.ultimaRepeticion) > HOLD_REPETIR_MS) {
    accion = true;
    b.ultimaRepeticion = ahora;
  }

  return accion;
}

// ---------- Pantalla LCD ----------
void actualizarLCD() {
  char linea0[17];
  char linea1[17];

  snprintf(linea0, sizeof(linea0), " Posicion Servo ");
  snprintf(linea1, sizeof(linea1), "   %3d deg     ", angulo);

  lcd.setCursor(0, 0);
  lcd.print(linea0);
  lcd.setCursor(0, 1);
  lcd.print(linea1);
}

// ---------- Mover Servo ----------
void cambiarValor(int delta) {
  angulo = constrain(angulo + delta, MIN_ANGULO, MAX_ANGULO);
  servo.write(angulo);
  actualizarLCD();
  Serial.printf("Angulo: %d deg\n", angulo);
}

// ---------- Setup y Loop ----------
void setup() {
  Serial.begin(115200);

  pinMode(BTN_UP,   INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);

  servo.setPeriodHertz(50);
  servo.attach(SERVO_PIN, 500, 2400);

  Wire.begin(LCD_SDA, LCD_SCL);
  lcd.init();
  lcd.backlight();

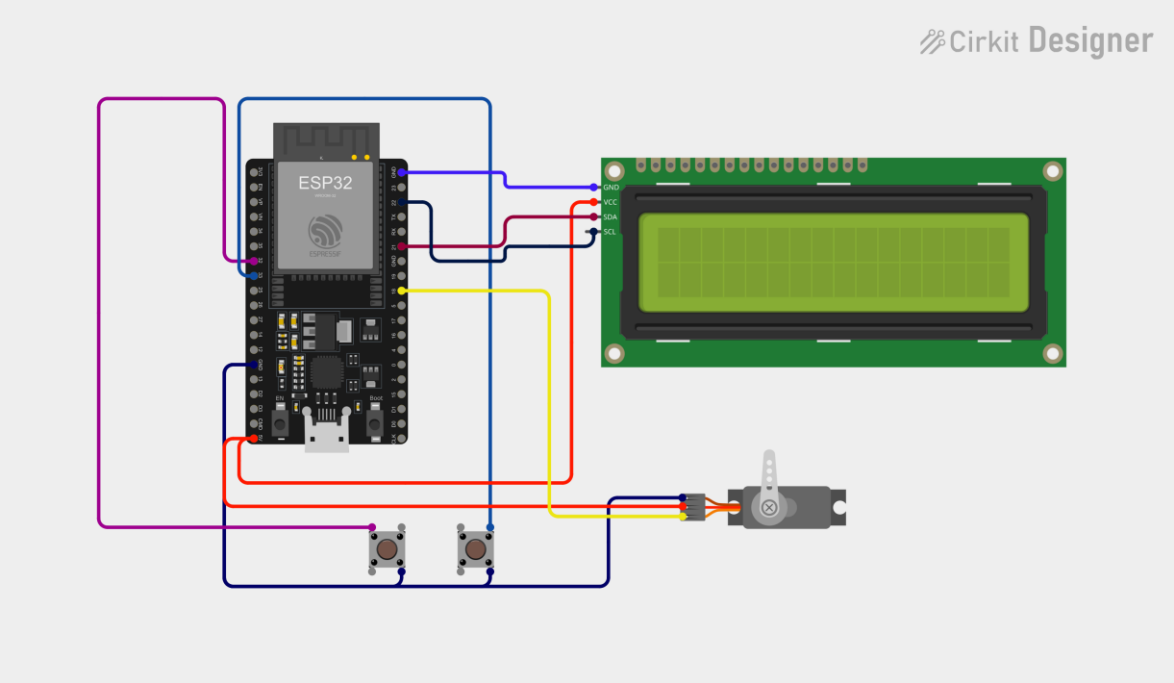
  servo.write(angulo);
  actualizarLCD();
}

void loop() {
  // Botón 1 (GPIO 32): Subir ángulo
  if (leerBoton(bUp)) {
    cambiarValor(+1);
  }
```

```
  // Botón 2 (GPIO 33): Bajar ángulo
  if (leerBoton(bDown)) {
    cambiarValor(-1);
  }

}
```

Repositorio Diagrama



Resultados

La ejecución y prueba experimental de la práctica arrojaron resultados totalmente satisfactorios y acordes con los objetivos planteados. La luz del foco pudo controlarse de manera correcta, fluida y precisa mediante el código implementado y el acoplamiento físico realizado. Al interactuar con los pulsadores para mover el servomotor SG90 hacia adelante o hacia atrás, el movimiento rotativo transmitido directamente a la perilla del dimmer reguló de forma gradual la potencia efectiva entregada al foco, permitiendo observar una transición estable desde la mínima

iluminación hasta el brillo máximo de la bombilla. Asimismo, la pantalla LCD respondió de inmediato reflejando sin interrupciones los grados actuales del servomotor, y el algoritmo de lectura de botones evitó falsos contactos por rebote mecánico, logrando una operación sumamente estable y sin bloqueos durante todo el desarrollo de las pruebas.

Conclusión

La realización de esta práctica permitió consolidar los conocimientos teóricos y prácticos relacionados con el control de sistemas electromecánicos, el manejo de actuadores de posicionamiento mediante señales PWM y la integración de microcontroladores modernos como el ESP32 en la automatización de procesos de potencia. Se demostró de manera empírica que es posible lograr un control de iluminación altamente funcional y regulable mediante el uso ingenioso de un acoplamiento mecánico entre un servomotor y la perilla de un dimmer convencional de corriente alterna. El diseño del firmware optimizado para la gestión de entradas de usuario y la visualización en tiempo real garantizó la robustez, precisión y alta eficiencia operativa del prototipo, cumpliendo a cabalidad con todos los requerimientos de la práctica.

Referencias APA

- Espressif Systems. (2025). *ESP32 Series Datasheet: Technical Specification*. Espressif Systems.
- SparkFun Electronics. (2024). *Standard Servo Motor Hookup Guide and Principles of Operation*. SparkFun Reference Documentation.
- National Electrical Manufacturers Association. (2023). *Standard for AC Dimmers and Power Control Devices*. NEMA Standards Publication.
- SparkFun Electronics. (2024). *I2C LCD Hookup Guide and Digital Input Principles*. SparkFun Reference Documentation.
- Ogata, K. (2010). *Ingeniería de Control Moderna* (5ª ed.). Pearson Educación.